

microGPT — Lab 1

Türkçe İsim Üretimi: Tokenizasyon, Eğitim ve Çıkarım

Andrej Karpathy'nin microGPT uygulaması üzerine pratik çalışmalar

Bu laboratuvar çalışmasında, Andrej Karpathy'nin sıfırdan yazdığı microGPT uygulamasını inceleyecek ve üç farklı alıştırma üzerinde çalışacaksınız. Kod; öz-farklılaşma (autograd), transformer mimarisi, Adam optimizörü ve karakter düzeyinde tokenizasyonun tamamını harici kütüphane kullanmadan gerçekleştirmektedir.

Ön Gereksinimler

Python 3.x kurulu olmalıdır. microgpt.py ve isimler.txt dosyaları proje kök dizininde bulunmalıdır. Harici kütüphane gerekmemektedir.

Proje Dosya Yapısı (Başlangıç)

```
microgpt-lab/
├── microgpt.py      # Orijinal tek-dosya uygulama (değiştirmeyin)
├── isimler.txt      # 2 446 Türkçe isim içeren veri seti
├── lab1.pdf         # Bu doküman
└── lab1/           # Oluşturacağınız dosyalar buraya gelecek
```

Alıştırma 1: Token Analizi — isimler.txt ile Kaç Farklı Token Var?

microGPT, karakter düzeyinde bir tokenizasyon kullanmaktadır. Her benzersiz karakter (harf, boşluk vb.) bir token kimliğine (ID) karşılık gelir. Orijinal kodda veri 'input.txt' dosyasından okunmaktadır. Bu alıştırma 'isimler.txt' dosyasındaki Türkçe isimleri kullanacaksınız.

Yapmanız Gerekenler

- microgpt.py dosyasındaki veri yükleme satırını isimler.txt'i okuyacak şekilde değiştirin.
- Programı çalıştırın ve vocab size değerini not edin.
- Hangi karakterlerin token olarak kullanıldığını listeleyin.

Değiştirilmesi Gereken Kod Satırı

```
# Orijinal
docs = [line.strip() for line in open('input.txt') if line.strip()]

# Değiştirin
docs = [line.strip() for line in open('isimler.txt') if line.strip()]
```

isimler.txt Veri Seti Hakkında

```
Toplam isim sayısı : 2 446
Örnek isimler      : JALE, ALİ, MAHMUT, MANSUR KÜRŞAD, GAMZE, MİRAÇ, YÜCEL ...
Özellikler         : Büyük harf, Türkçe özel karakterler (Ç Ö Ü Ğ İ Ş),
                    boşluklu bileşik isimler (BEDRİYE MÜGE vb.)
```

Cevaplamanız Gereken Sorular

1. Programı çalıştırdığınızda kaç farklı token (karakter) bulunmaktadır? BOS dahil vocab_size nedir?
2. Türkçe isimlerde hiç kullanılmayan Latin harfleri hangileridir? Bu harflerin neden vocabulary'de yer almadığını açıklayın.
3. isimler.txt dosyasında büyük harfli isimler ile birlikte küçük harfli isimler kullansaydık vocab_size nasıl değişirdi?
4. BOS (Beginning of Sequence) tokeninin görevi nedir? Modelin eğitiminde ne işe yarar?

Alıştırma 2: Modüler Yapı — Eğitimi, Modeli ve Testi Ayırma

Orijinal microgpt.py dosyası eğitim ve çıkarımı tek dosyada birleştirmektedir. Bu alıştırma kodu iki ayrı dosyaya bölecek ve model ağırlıklarını pickle ile kaydedip yükleyeceksiniz.

Oluşturmanız Gereken Dosyalar

```
lab1/
├── train.py      # Modeli eğitir, ağırlıklarını 'model.pkl' olarak kaydeder
├── model.pkl     # train.py tarafından oluşturulur
└── test.py      # model.pkl'ı yükler, yeni Türkçe isimler üretir
```

train.py — İskelet Yapı

```
import os, math, random, pickle
random.seed(42)

# 1. Veri yükleme
docs = [line.strip() for line in open('isimler.txt') if line.strip()]
random.shuffle(docs)

# 2. Tokenizasyon
uchars = sorted(set(''.join(docs)))
BOS = len(uchars)
vocab_size = len(uchars) + 1

# 3. Value sınıfı, model parametreleri ve mimari (microgpt.py'den)

# 4. Eğitim döngüsü (microgpt.py'deki for step ... bloğu)

# 5. Modeli kaydet
save_data = {
    'uchars': uchars, 'BOS': BOS, 'vocab_size': vocab_size,
    'state_dict_data': {k: [[p.data for p in row] for row in mat]
                        for k, mat in state_dict.items()}
}
with open('model.pkl', 'wb') as f: pickle.dump(save_data, f)
```

test.py — İskelet Yapı

```
import random, math, pickle

# 1. Modeli yükle
with open('model.pkl', 'rb') as f:
    save_data = pickle.load(f)

uchars = save_data['uchars']
BOS = save_data['BOS']
vocab_size = save_data['vocab_size']
```

```
# 2. Value sınıfını tanımla, state_dict_data'yı Value nesnelere dönüştür

# 3. Çıkarım – 20 yeni isim üret
temperature = 0.5
for sample_idx in range(20):
    ...
```

Dikkat — pickle ve Value Sınıfı

`pickle.dump` doğrudan Value nesnelerini serileştiremez (iç içe computation graph nedeniyle). Yalnızca `'data'` sayısal değerlerini kaydedin. `test.py`'de her değeri `Value(float_value)` olarak yeniden sarmalayın.

Cevaplamanız Gereken Sorular

1. `train.py` ve `test.py` ayrımının gerçek dünya ML projelerinde sağladığı avantajlar nelerdir?
2. `train.py` çalıştırdıktan sonra `model.pkl` dosyasının boyutu nedir (bytes)? Bu boyutu model parametre sayısı ile ilişkilendirebilir misiniz?
3. Modelin kaç parametresi bulunmaktadır? (İpucu: `'num params'` çıktısına bakın.)
4. `random.seed(42)` olmadan `test.py`'yi iki kez çalıştırsaydınız aynı isimleri üretir miydi? Neden?

Alıştırma 3: Temperature Parametresi — Yaratıcılık ve Belirlilik Dengesi

Temperature, dil modellerinin çıktısını kontrol eden kritik bir hiperparametredir. Bu alıştırmada temperature değerini komut satırı argümanı olarak alan ayrı bir dosya yazacak ve farklı değerlerin çıktı üzerindeki etkisini inceleyeceksiniz.

Oluşturmanız Gereken Dosya ve Kullanımı

```
# Dosya: lab1/test_temperature.py

python test_temperature.py --temperature 0.1
python test_temperature.py --temperature 0.5
python test_temperature.py --temperature 1.0
python test_temperature.py --temperature 2.0
```

test_temperature.py — İskelet Yapı

```
import argparse, random, math, pickle

parser = argparse.ArgumentParser(description='microGPT isim üretici')
parser.add_argument('--temperature', type=float, default=0.5,
                    help='Örnekleme sıcaklığı (0 < t)')
args = parser.parse_args()
temperature = args.temperature

# Model yükleme kodu (test.py ile aynı)
# ...

# Temperature ölçeklemeli softmax
probs = softmax([l / temperature for l in logits])
```

Temperature'ın Matematiksel Etkisi

$P(\text{token}_i) = \exp(\text{logit}_i / T) / \sum \exp(\text{logit}_j / T)$

$T \rightarrow 0$: Olasılık tek token'da yoğunlaşır (greedy seçim)
 $T = 0.5$: Yüksek olasılıklı tokenlar ön plana çıkar
 $T = 1.0$: Ham softmax (değişiklik yok)
 $T > 1.0$: Dağılım düzleşir, tüm tokenlar benzer olasılık kazanır

Görev: Farklı Temperature Değerlerini Deneyin ve Tabloyu Doldurun

train.py ile eğitilmiş modeli kullanarak aşağıdaki tabloyu kendiniz doldurunuz:

Temperature	Örnek 1	Örnek 2	Örnek 3	Gözlemleriniz
0.1				
0.5				
1.0				
2.0				

Cevaplamanız Gereken Sorular

1. $T=0.1$ ile $T=2.0$ arasında üretilen isimler arasındaki en belirgin fark nedir?
2. Hangi temperature değerinde üretilen isimler gerçek Türkçe isimlere en çok benziyor? Neden?
3. `random.seed(42)` ile aynı temperature için her çalıştırmada aynı sonuçlar mı üretilir? Neden?
4. Temperature parametresi ile model ağırlıkları arasındaki temel fark nedir? Hangisi eğitilden etkilenir?
5. Veri seti genişletme (data augmentation) amacıyla isim üretecek olsanız hangi temperature değerini seçerdiniz? Gerekçenizi açıklayın.

Özet ve Teslim Gereksinimleri

Bu laboratuvar çalışmasını tamamladığınızda aşağıdaki yapıya sahip olmalısınız:

```
lab1/
├── train.py          # Alıştırma 2
├── test.py           # Alıştırma 2
├── test_temperature.py # Alıştırma 3
├── model.pkl         # train.py tarafından üretilir
└── rapor.pdf         # Soruların cevapları + tablo + gözlemler
```

Değerlendirme

Alıştırma 1 — 25 puan	isimler.txt doğru yükleniyor, vocab_size hesaplanmış, 4 soru yanıtlanmış.
Alıştırma 2 — 35 puan	train.py ve test.py çalışıyor, model.pkl oluşturuluyor, 4 soru yanıtlanmış.
Alıştırma 3 — 30 puan	test_temperature.py --temperature argümanını alıyor, tablo doldurulmuş, 5 soru yanıtlanmış.
Rapor kalitesi — 10 puan	Cevaplar açık, teknik olarak doğru ve kendi cümlelerinle yazılmış.

Hatırlatma

microgpt.py dosyasını değiştirmeyin — referans olarak kullanacaksınız.

Tüm yeni dosyalarınızı lab1/ klasörüne kaydedin.

pickle kullanırken yalnızca .data sayısal değerlerini serileştirin.